



UNIVERSIDAD AUTÓNOMA DE CAMPECHE
Ingeniería en Tecnología de Software



Materia

Concurrencia y Paralelismo

Maestro

Sergio Noh Puch

Trabajo

Práctica 01 – Sección Crítica

Alumna

Laura Lucia Jaramillo Yah

Matrícula

68636

Fecha

28/08/2026

INTRODUCCIÓN

En los sistemas operativos actuales, la programación concurrente por medio de **hilos (threads)** permite realizar varias tareas al mismo tiempo ayudando a mejorar los tiempos de procesamiento. Sin embargo, debido a que los hilos comparten el mismo espacio de memoria, pueden presentarse problemas cuando dos o más intentan modificar una misma variable global de manera simultánea.

El objetivo de esta práctica es crear un contador global compartido entre varios hilos utilizando la biblioteca **POSIX Threads (pthread)**. También se busca aplicar un mecanismo de sincronización como el **mutex** para evitar errores en los datos y analizar el funcionamiento del código en lenguaje ensamblador ayudando a identificar las instrucciones del procesador que participan en la modificación de la memoria.



CÓDIGO INICIAL

Antes de implementar el pseudocódigo, el código inicial crea dos hilos, llamados **thread_one** y **thread_two**, mediante las rutinas **thread_routine** y **thread_routine_two**. En el funcionamiento original del programa, la función **main** recibe el valor ingresado por el usuario desde la consola y lo convierte en un número entero mediante **counter = atoi(argv[1])**, después crea los dos hilos y les pasa la dirección de esta variable.

Dentro de cada rutina se utiliza **pthread_mutex_lock(&mutex)** antes de comenzar el ciclo for. Debido a esto, el primer hilo que obtiene el **mutex** mantiene el control del recurso y escribe todas sus líneas en el archivo readme.txt. El **mutex** se libera únicamente cuando termina todas sus iteraciones, por lo que el segundo hilo debe esperar y no existe un paralelismo efectivo entre ambos.

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void *thread_routine(void *arg){
    int num_line = *((int*)arg);
    int fd;
    char buf[] = "New line \n";

    printf("starting thread...");

    pthread_mutex_lock (&mutex);
    for (int i = 0; i < num_line; i++) {
        fd = open("/home/lucyjy/CyP_prueba/readme.txt", O_WRONLY | O_APPEND);
        write(fd, buf, sizeof(buf)-1);
        close(fd);
    }
    pthread_mutex_unlock (&mutex);
}
```

```
void *thread_routine_two(void *arg){
    int num_line = *((int*)arg);
    int fd;
    char buf[] = "Lucy \n";

    printf("starting thread two... \n");

    pthread_mutex_lock (&mutex);
    for (int i = 0; i < num_line; i++) {
        fd = open("/home/lucyjy/CyP_prueba/readme.txt", O_WRONLY | O_APPEND);
        write(fd, buf, sizeof(buf)-1);
        close(fd);
    }
    pthread_mutex_unlock (&mutex);
}
```

```

int main (int argc, char const *argv[]) {
    int counter = 0;
    pthread_t thread_one;
    pthread_t thread_two;
    counter = atoi(argv[1]);

    if (0!=pthread_create(&thread_one, NULL, thread_routine, &counter))
        return -1;

    if (0!=pthread_create(&thread_two, NULL, thread_routine_two, &counter))
        return -1;

    pthread_join(thread_one, NULL);
    pthread_join(thread_two, NULL);

    return 0;
}

```



IMPLEMENTACIÓN DEL PSEUDOCÓDIGO Y SINCRONIZACIÓN

Para integrar las instrucciones solicitadas y conseguir una ejecución concurrente y segura, modifiqué el código para trabajar de manera dinámica con la variable **global_counter**, cuyo valor inicial es 20. En esta nueva versión incluí la biblioteca **<string.h>** para el manejo de cadenas de texto. También agregué la condición **if (argc < 2)** dentro de la función **main**, la cual comprueba que el usuario haya ingresado el argumento necesario. De esta manera, el programa puede mostrar la forma correcta de utilizarlo y evita intentar acceder a una posición de memoria inválida.

A diferencia del código original, la instrucción **pthread_mutex_lock(&mutex)** la coloqué dentro del ciclo **for**. Este cambio reduce la sección crítica solamente al momento en que se modifica la variable global y se escribe el mensaje en el archivo. Gracias a esto, los dos hilos pueden competir de manera coordinada por el **mutex** en cada iteración, permitiendo que sus operaciones se intercalen durante la ejecución.

Finalmente, utilicé la función **snprintf** para crear los mensajes que se guardan en el archivo. El hilo **thread_routine** aumenta el valor de la variable y registra cada operación con el formato **"Thread one: X + 1 = Y"**, mientras que **thread_routine_two** disminuye su valor y utiliza el formato **"Thread two: X - 1 =**

Y". Así, cada modificación queda registrada correctamente y de manera coherente dentro del archivo **readme.txt**.



CONDICIÓN DE carrera

Una **condición de carrera** se presenta en la programación concurrente cuando dos o más hilos acceden y modifican al mismo tiempo una variable o recurso compartido sin utilizar un mecanismo de sincronización adecuado. Debido a que el planificador de tareas del sistema operativo puede detener o alternar la ejecución de los hilos en cualquier momento, el resultado final depende del orden impredecible en el que se realicen las instrucciones.

Cuando ocurre una condición de carrera, el programa deja de tener un comportamiento determinista. Esto quiere decir que, aunque se ejecute varias veces con los mismos datos de entrada, puede producir resultados distintos en cada ocasión. Como consecuencia, pueden perderse modificaciones o generarse valores incorrectos e inconsistentes dentro del sistema.

sección crítica

La sección crítica es la parte de un programa en la que se accede o modifica algún recurso compartido, como una variable global, un archivo o un dispositivo de hardware. Para mantener la integridad de los datos en programas con varios hilos, esta sección debe cumplir con los principios de exclusión mutua, progreso y espera limitada. La exclusión mutua indica que, mientras un hilo se encuentra dentro de la sección crítica, ningún otro puede entrar y modificar el mismo recurso hasta que el primero termine y salga de ella.

En C, la exclusión mutua con hilos POSIX se puede implementar mediante un cerrojo llamado **mutex**, representado por el tipo de dato **pthread_mutex_t**. De manera sencilla, puede entenderse como un candado digital que mantiene un estado de bloqueo. Cuando un hilo necesita acceder a la sección crítica, utiliza la función **pthread_mutex_lock()**. Si el **mutex** está disponible, el hilo lo adquiere de manera atómica y continúa su ejecución.

En cambio, si ya se encuentra ocupado por otro hilo, el hilo que intenta acceder queda en espera hasta que el recurso sea liberado.

USO DEL MUTEX

Durante esta práctica se utilizó la macro **PTHREAD_MUTEX_INITIALIZER** para inicializar el **mutex** de manera estática antes de crear los hilos. Dentro de las funciones correspondientes a cada hilo, el **mutex** protegió tanto el acceso a la variable global **global_counter** como la escritura en el archivo **readme.txt**. En cada iteración del ciclo se ejecutaba **pthread_mutex_lock(&mutex)** antes de modificar la variable compartida, asegurando que solo un hilo trabajara con ella a la vez. Al finalizar la sección crítica, se utilizaba **pthread_mutex_unlock(&mutex)** para liberar el cerrojo y permitir que el otro hilo pudiera competir nuevamente por el acceso al recurso.



ANÁLISIS DEL CÓDIGO ensamblador

Al compilar el archivo con el comando **gcc -S P1_Contador.c**, se generó su representación en lenguaje ensamblador para la arquitectura x86_64. A partir de este análisis pude observar que una instrucción sencilla escrita en C se divide en varias operaciones a nivel del procesador, por lo que no se ejecuta como una acción única e indivisible.

Teniendo esto en cuenta, al revisar las instrucciones en ensamblador encontré el siguiente funcionamiento:

Incremento en **thread_routine (global_counter++)**

1. **movl global_counter(%rip), %eax:** Obtiene el valor actual de **global_counter** desde la memoria y lo copia en el registro **%eax** del procesador.
2. **addl \$1, %eax:** Aumenta en 1 el valor que se encuentra almacenado en el registro **%eax**.

3. **movl %eax, global_counter(%rip):** Guarda nuevamente en la memoria global el valor actualizado del registro %eax.

Decremento en thread_routine_two (global_counter--)

1. **movl global_counter(%rip), %eax:** Obtiene el valor de global_counter desde la memoria y lo carga en el registro %eax.
2. **subl \$1, %eax:** Disminuye en 1 el valor almacenado dentro del registro %eax.
3. **movl %eax, global_counter(%rip):** Escribe el resultado actualizado del registro %eax en la posición de memoria correspondiente a global_counter.

Demostración de la no atomicidad

La separación de la operación en tres instrucciones (**movl** → **addl/subl** → **movl**) permite observar que el procesador realiza un proceso de lectura, modificación y escritura. Sin la protección del mutex, un hilo podría ser interrumpido después de leer el valor de la variable y antes de guardar el resultado. Durante ese momento, el segundo hilo podría leer el mismo valor desactualizado y posteriormente sobrescribir la modificación realizada por el primero, ocasionando resultados incorrectos y pérdida de coherencia en los datos.

```
movl    (%rax), %eax
movl    %eax, -120(%rbp)
movl    $0, -124(%rbp)
movl    global_counter(%rip), %eax
movl    global_counter(%rip), %edx
movl    %ecx, %r8d
movl    %edx, %ecx
movl    $100, %esi
movl    $0, %eax
movl    global_counter(%rip), %eax
movl    %eax, global_counter(%rip)
movl    $420, %edx
movl    $1089, %esi
movl    $0, %eax
movl    %eax, -116(%rbp)
movl    -116(%rbp), %eax
movl    %eax, %edi
movl    -116(%rbp), %eax
movl    %eax, %edi
movl    -124(%rbp), %eax
movl    $0, %eax
movl    %edi, -36(%rbp)
movl    $0, %eax
movl    ($rax), %eax
movl    %eax, -120(%rbp)
movl    $0, -124(%rbp)
movl    global_counter(%rip), %eax
movl    global_counter(%rip), %edx
movl    %ecx, %r8d
movl    %edx, %ecx
movl    $100, %esi
movl    $0, %eax
movl    global_counter(%rip), %eax
movl    %eax, global_counter(%rip)
movl    $420, %edx
```

```
movl    $100, %esi
movl    $0, %eax
movl    %eax, -116(%rbp)
movl    -116(%rbp), %eax
movl    %eax, %edi
movl    -116(%rbp), %eax
movl    %eax, %edi
movl    -124(%rbp), %eax
movl    $0, %eax
movl    %edi, -36(%rbp)
movl    $0, %eax
movl    $-1, %eax
movl    %eax, -28(%rbp)
movl    $0, %esi
movl    $-1, %eax
movl    $0, %esi
movl    $-1, %eax
movl    $0, %esi
movl    $0, %esi
movl    $0, %eax
```

```
lucyjjy@DESKTOP-TDQ5Q7L:~/CyP_prueba/Practica1$ grep -E addl P1_Contador.s
    addl    $1, %eax
    addl    $1, -124(%rbp)
    addl    $1, -124(%rbp)
lucyjjy@DESKTOP-TDQ5Q7L:~/CyP_prueba/Practica1$ grep -E subl P1_Contador.s
    subl    $1, %eax
```

